```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

1

```
import os
import shutil
import random
import json
```

CONFIGURATION — update these two paths only

```
SOURCE_DIR = "/content/drive/MyDrive/Project work/Dataset/lgg-mri-segmentation/kaggle_3m"
OUTPUT_DIR = "/content/drive/MyDrive/Project work/Dataset/lgg-mri-segmentation/lgg_structured"
# ────────────────────────────────────────────

RANDOM_SEED = 42
```

```
print(os.path.exists(SOURCE_DIR))
```

```
True
```

Step 1 — get all patient folders (skip non-folder items like desktop.ini)

```
all_patients = sorted([
    f for f in os.listdir(SOURCE_DIR)
    if os.path.isdir(os.path.join(SOURCE_DIR, f))
])
print(f"Total patients found: {len(all_patients)}")
```

```
Total patients found: 110
```

```
# Step 2 — shuffle and split
random.seed(RANDOM_SEED)
shuffled = all_patients.copy()
random.shuffle(shuffled)

train_end = int(len(shuffled) * 0.80)
val_end   = train_end + int(len(shuffled) * 0.10)

train_patients = shuffled[:train_end]
val_patients   = shuffled[train_end:val_end]
test_patients  = shuffled[val_end:]

print(f"Train: {len(train_patients)} | Val: {len(val_patients)} | Test: {len(test_patients)}")
```

```
Train: 88 | Val: 11 | Test: 11
```

```
# Step 3 — create output folders
for split in ["train", "val", "test"]:
    for folder in ["images", "masks"]:
        os.makedirs(os.path.join(OUTPUT_DIR, split, folder), exist_ok=True)

print("Output folders created.")
```

```
Output folders created.
```

```
# Step 4 — copy files
# Accepts both .tif and .png just in case
VALID_EXTENSIONS = (".tif", ".tiff", ".png")

def copy_split(patients, split_name):
    img_count  = 0
    mask_count = 0

    for patient in patients:
        patient_folder = os.path.join(SOURCE_DIR, patient)
        for filename in os.listdir(patient_folder):

            # skip non-image files like desktop.ini
            if not filename.lower().endswith(VALID_EXTENSIONS):
                continue

            src = os.path.join(patient_folder, filename)

            if "_mask" in filename:
                dst = os.path.join(OUTPUT_DIR, split_name, "masks", filename)
                mask_count += 1
            else:
                dst = os.path.join(OUTPUT_DIR, split_name, "images", filename)
                img_count += 1

            shutil.copy2(src, dst)

    print(f"[{split_name}] Images: {img_count} | Masks: {mask_count}")

copy_split(train_patients, "train")
copy_split(val_patients,   "val")
copy_split(test_patients,  "test")
```

```
[train] Images: 3133 | Masks: 3133
[val] Images: 409 | Masks: 409
[test] Images: 387 | Masks: 387
```

```
# Step 5 — save patient split record
split_record = {
    "train": train_patients,
    "val":   val_patients,
    "test":  test_patients
}
with open(os.path.join(OUTPUT_DIR, "patient_split.json"), "w") as f:
    json.dump(split_record, f, indent=4)

print("patient_split.json saved.")
```

```
patient_split.json saved.
```

```
# Step 6 — final verification
print("\n===== VERIFICATION =====")
for split in ["train", "val", "test"]:
    imgs  = len(os.listdir(os.path.join(OUTPUT_DIR, split, "images")))
    masks = len(os.listdir(os.path.join(OUTPUT_DIR, split, "masks")))
    match = "✓" if imgs == masks else "✗ MISMATCH"
    print(f"  {split:5s} → images: {imgs} | masks: {masks}  {match}")

print("========================")
print("Done.")
```

```
===== VERIFICATION =====
  train → images: 3133 | masks: 3133  ✓
  val   → images: 409 | masks: 409  ✓
  test  → images: 387 | masks: 387  ✓
========================
Done.
```

***djkjwd ***

Double-click (or enter) to edit